OpenCode, l'alternative open source à Claude Code

Comprendre et utiliser les agents IA pour développer

Projet FYC de [Prénom Nom], [Prénom Nom], [Prénom Nom] et [Prénom Nom]
encadré par [Nom du mentor]
ESGI, 4e année, 2026-2027
(Fiche scientifique préparatoire)

Contexte et état de l'art

Depuis quelques années, les outils d'aide au développement fondés sur l'intelligence artificielle se sont largement diffusés. Les premiers outils d'autocomplétion se limitaient à terminer la ligne en cours. L'arrivée des grands modèles de langage (LLM), construits sur l'architecture Transformer [Vaswani et al., 2017], a permis de générer des fonctions entières à partir du contexte d'un fichier, comme le fait GitHub Copilot à partir du modèle Codex [Chen et al., 2021]. Le réglage de ces modèles par retour humain [Ouyang et al., 2022] a ensuite rendu possible un usage conversationnel, popularisé par ChatGPT fin 2022.

Ces usages restent cependant passifs : le modèle propose, et c'est le développeur qui exécute, teste et recopie le code. Les travaux sur les agents ont montré qu'un modèle peut alterner raisonnement et actions, en appelant des outils puis en observant leur résultat [Yao et al., 2023, Schick et al., 2023]. C'est sur ce principe que reposent les agents de développement apparus en 2025, comme Claude Code [Anthropic, 2025] : un modèle, un ensemble d'outils (lecture et écriture de fichiers, terminal, recherche dans le code) et une boucle qui s'exécute jusqu'à la fin de la tâche. Leur capacité à résoudre de vrais tickets issus de projets open source est aujourd'hui mesurée par des bancs d'essai comme SWE-bench [Jimenez et al., 2024].

L'efficacité d'un agent dépend fortement du contexte qu'il reçoit. Les modèles exploitent moins bien une information placée au milieu d'un long contexte [Liu et al., 2024], ce qui pousse à sélectionner les informations utiles plutôt qu'à tout fournir. Pour apporter des données externes au modèle, deux approches coexistent : la génération augmentée par récupération, ou RAG [Lewis et al., 2020], et le Model Context Protocol (MCP), un standard ouvert qui permet à l'agent d'accéder lui-même à des outils et à des sources de données [Anthropic, 2024]. Le serveur MCP Context7 [Upstash, 2025] en est un exemple : il fournit à l'agent la documentation à jour des bibliothèques utilisées dans le projet.

Les agents de développement les plus utilisés sont pour la plupart propriétaires et liés à un seul fournisseur de modèles. OpenCode [SST, 2025] propose une alternative : un agent en terminal publié sous licence MIT, compatible avec de nombreux fournisseurs de modèles ainsi qu'avec des modèles exécutés en local.

Sujet et problématique

Notre projet porte sur OpenCode et, plus largement, sur l'usage des agents IA dans le développement logiciel. Il prend la forme d'un cours vidéo destiné à des étudiants en développement web de niveau Mastère.

Le choix d'un agent ne se limite pas à la qualité du code qu'il produit. Il engage aussi le coût d'utilisation, facturé au nombre de tokens traités, la confidentialité du code envoyé au fournisseur et la dépendance envers un éditeur. Un outil fermé comme Claude Code bénéficie d'une intégration poussée avec les modèles de son éditeur. Un outil ouvert laisse le choix du

modèle, mais ses résultats dépendent alors de ce choix. Enfin, mal utilisé, un agent peut produire du code faux ou exécuter des actions non souhaitées.

Notre problématique est donc la suivante : OpenCode est-il une alternative crédible à Claude Code, et comment l'utiliser efficacement et en sécurité sur un projet réel ?

Méthode

Nous abordons cette question en trois temps. D'abord, nous construisons les notions nécessaires pour comprendre ce qu'est un agent. Le cours part du fonctionnement d'un LLM (prédiction du token suivant, entraînement, limites), puis présente la boucle agentique (figure 1), le rôle du contexte et le protocole MCP. Nous présentons ensuite Claude Code et OpenCode, ainsi que l'architecture client-serveur de ce dernier (figure 2).

Nous passons ensuite à la pratique sur un projet de taille modeste : une API Node.js/Express accompagnée de tests. L'apprenant installe et configure OpenCode, puis réalise un cas pratique en trois parties : comprendre un projet qu'il ne connaît pas, y ajouter une fonctionnalité (la pagination d'un endpoint), puis corriger un bug signalé par un test. Chaque partie est accompagnée d'une correction en vidéo.

Enfin, nous comparons les deux outils à partir de leur documentation officielle, sur des critères qui comptent pour un développeur ou une entreprise : licence, modèles utilisables, coût, devenir des données et maturité (tableau 1). Le cours se termine par des bonnes pratiques d'usage des agents, notamment sur la relecture du code produit et la sécurité.

Résultats

Le cours se compose de douze vidéos courtes, pour une durée totale d'environ 1 h 35, et d'un cas pratique en trois parties. À la fin du cours, l'apprenant doit être capable d'expliquer le fonctionnement d'un agent IA, d'installer et de configurer OpenCode avec le fournisseur de son choix, puis de l'utiliser sur un projet existant en gardant la main sur les modifications proposées.

La comparaison montre qu'OpenCode est moins abouti que Claude Code, dont l'intégration avec les modèles d'Anthropic est très poussée. En revanche, il laisse au développeur le choix du modèle, la maîtrise des coûts et la possibilité de garder ses données en local. Il constitue donc une alternative crédible lorsque ces critères comptent, à condition de l'utiliser avec méthode.

Aller plus loin

OpenCode évolue rapidement, et certaines commandes présentées dans le cours peuvent changer d'une version à l'autre. La qualité des résultats dépend aussi beaucoup du modèle branché sur OpenCode.

L'autonomie d'un agent pose enfin des questions de sécurité. Un agent qui lit des fichiers, des pages web ou des tickets peut y trouver des instructions malveillantes, une attaque appelée injection de prompt indirecte [Greshake et al., 2023]. Un prolongement de ce travail serait d'étudier les mécanismes de permission et d'isolation proposés par ces outils, et la façon de les configurer dans un contexte d'entreprise.

Références

Anthropic (2024). Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol

Anthropic (2025). Claude Code overview. Documentation officielle. https://docs.claude.com/en/docs/claude-code/overview

Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. (2023). Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *AISec '23*, ACM.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *ICLR*.

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the ACL*, 12:157–173.

Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. In *NeurIPS*.

Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. In *NeurIPS*.

SST (2025). OpenCode, documentation officielle. https://opencode.ai/docs

Upstash (2025). Context7. https://context7.com

Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. In *NeurIPS*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*.

Annexes (figures et tableaux)

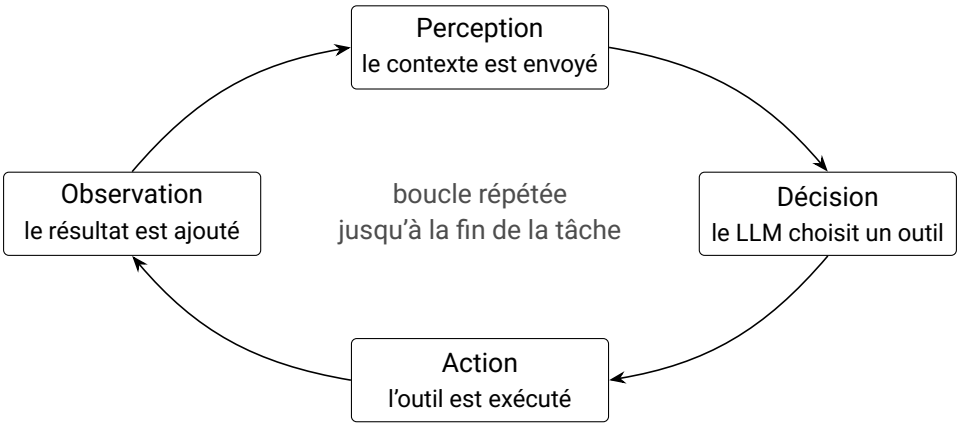

Figure 1 – Fonctionnement d'un agent IA : la boucle perception, décision, action, observation

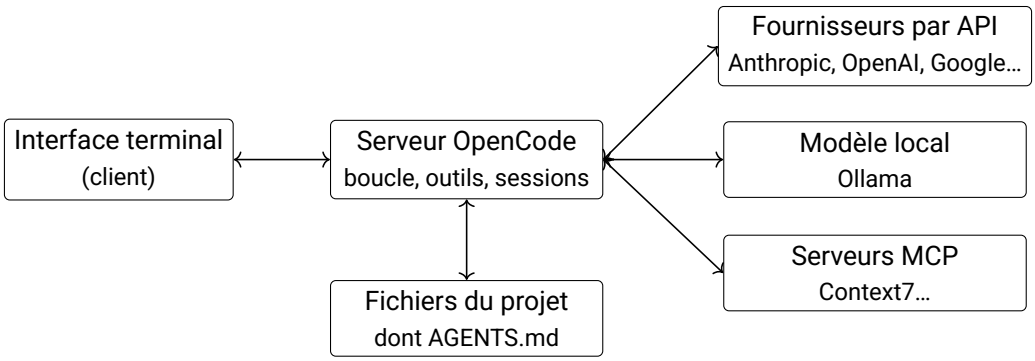

Figure 2 – Architecture d'OpenCode : un serveur local relié au modèle choisi, aux outils et au projet

Tableau 1 – Critères de comparaison entre Claude Code et OpenCode

Critère	Claude Code	OpenCode
Licence	Propriétaire	Open source (MIT)
Modèles	Modèles Anthropic uniquement	Nombreux fournisseurs, modèles locaux
Coût	Abonnement ou facturation à l'usage	Outil gratuit, modèle payé au fournisseur
Données	Envoyées à Anthropic	Fournisseur au choix, ou traitement local
Consignes projet	Fichier CLAUDE.md	Fichier AGENTS.md (format ouvert)
Maturité	Très aboutie	Récente, évolue rapidement